UnrealEngine5におけるメモリスキャンと任意関数呼び出し

ゲームアンチチートに学ぶ セキュリティ設計と攻防の舞台裏
by みずあめ

About Me

名前: 佐藤 良

年齢: 19歳

所属: 筑波大学情報学群情報科学類2年

X: @mizuameisgod

GitHub: @mizuamedesu

web: https://mizuame.works

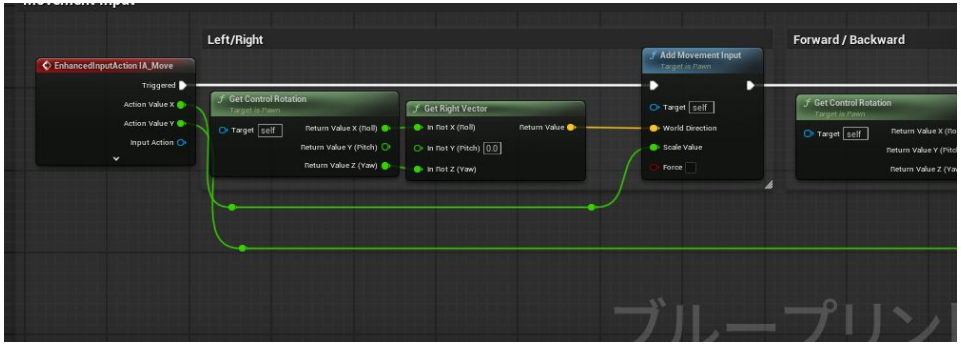
普段はゲームクライアントエンジニアとして、UnrealEngineやUnityをやってます

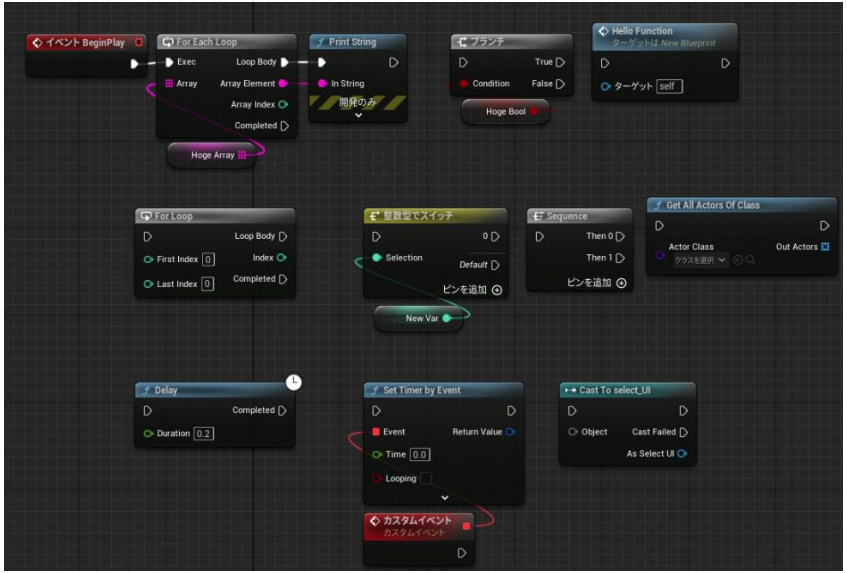
このスライドは
https://mizuame.works/slides/
で見れます

話すこと: UnrealEngineの話

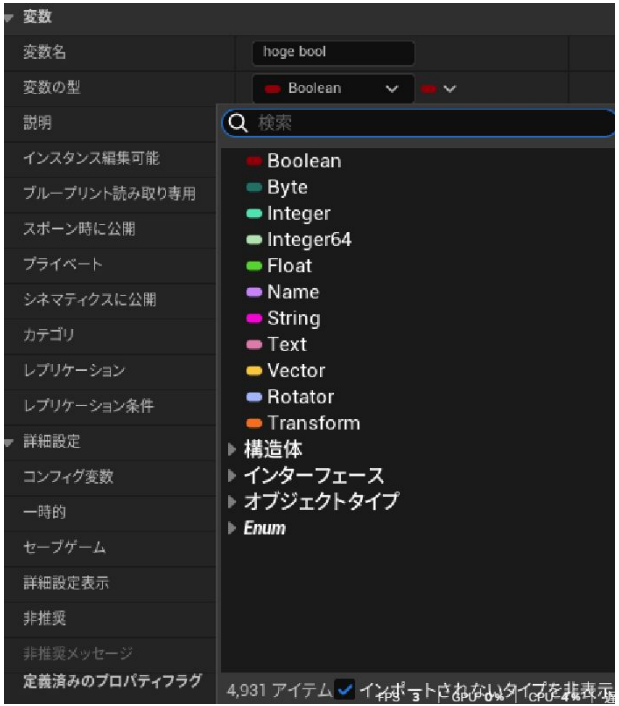
話さないこと: 難しいパソコンの話

UnrealEngineでロジックを書く方法はBPかC++





UnrealEngineでロジックを書く方法はBPかC++



Blueprint(通称BP)は、ビジュアルスクリプトなDSL。
コンパイル時にbytecodeになり、VMで実行される。
BPのVMは制御フローや変数アクセスは自前で処理するが、演算・IO等の実処理はすべてC++関数呼び出しに委譲している。
加算をしたいなら、加算をするC++の関数を呼ぶのような設計。
BlueprintFunctionLibraryとして自分でC++を書き拡張可能。

UnrealEngineでロジックを書く方法はBPかC++

```
61  {
70      }
71      }
72  }
73
74  void ARewindRPCTestCharacter::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
75  {
76      // Set up action bindings
77      if (UEnhancedInputComponent* EnhancedInputComponent = Cast<UEnhancedInputComponent>(PlayerInputComponent)) {
78
79          // Jumping
80          EnhancedInputComponent->BindAction(JumpAction, ETriggerEvent::Started, this, &ACharacter::Jump);
81          EnhancedInputComponent->BindAction(JumpAction, ETriggerEvent::Completed, this, &ACharacter::StopJumping);
82
83          // Moving
84          EnhancedInputComponent->BindAction(MoveAction, ETriggerEvent::Triggered, this, &ARewindRPCTestCharacter::Move);
85
86          // Looking
87          EnhancedInputComponent->BindAction(LookAction, ETriggerEvent::Triggered, this, &ARewindRPCTestCharacter::Look);
88      }
89      else
90      {
91          UE_LOG(LogTemplateCharacter, Error, TEXT("'%s' Failed to find an Enhanced Input component! This template is built to use the Enhanced Inpu
92      }
93  }
94
95  void ARewindRPCTestCharacter::Move(const FInputActionValue& Value)
96  {
97      // input is a Vector2D
98      FVector2D MovementVector = Value.Get<FVector2D>();
99
100     if (Controller != nullptr)
101     {
102         // find out which way is forward
103         const FRotator Rotation = Controller->GetControlRotation();
104         const FRotator YawRotation(0, Rotation.Yaw, 0);
105
106         // get forward vector
107         const FVector ForwardDirection = FRotationMatrix(YawRotation).GetUnitAxis(EAxis::X);
108
109         // get right vector
110         const FVector RightDirection = FRotationMatrix(YawRotation).GetUnitAxis(EAxis::Y);
111
112         // add movement
113         AddMovementInput(ForwardDirection, MovementVector.Y);
114         AddMovementInput(RightDirection, MovementVector.X);
115     }
```

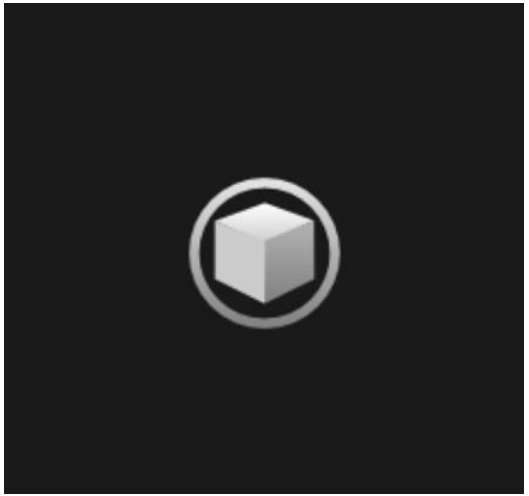
Unreal C++は標準C++に、Unrealエンジン独自の拡張を加えたもの。

拡張の多くは マクロ と UHT（Unreal Header Tool）によって実現されている。

UHTはUCLASS() UPROPERTY() UFUNCTION()などのマクロを元に自動生成コードを出力する。

神クラス、UObject

Unrealの全オブジェクトの基底クラス



- ガベージ コレクション(つまりエンジンが独自にGCを行う)
- 参照の更新
- リフレクション
- シリアル化
- デフォルト プロパティの変更の自動更新
- プロパティの自動初期化
- エディタの自動統合
- ランタイム時に使用可能な型情報
- ネットワークのレプリケーション

https://dev.epicgames.com/documentation/ja-jp/unreal-engine/objects-in-unreal-engine

リフレクション情報の実体

UClass — UCLASS()を付けたクラスに対して生成される。継承関係・関数・プロパティのリストを持つ。

UFunction — UFUNCTION()を付けた関数に対して生成される。UStructを継承しており、引数・戻り値をFPropertyのリストとして保持する。

FProperty — UPROPERTY()を付けたメンバ変数に対して生成される。型・サイズ・オブジェクト内オフセットを保持する。UE5からはUObjectを継承しない軽量なFFieldベースに変わっている。

他にもあるがメモリスキャンだとここら辺が知れると嬉しい

→これらはUHTによって自動生成されている

Blueprintの場合は？

C++の場合、UFUNCTION()等のマクロをUHTが解析し、ビルド時に自動生成コードを出力する。ゲーム起動時にその登録コードが実行され、UFunction・UClass・FPropertyのインスタンスがメモリ上に構築される。

BPの場合→エディタ上でBPをコンパイルするとBPコンパイラがノードグラフを解析し、UFunctionインスタンスと関連するFPropertyを生成する。その後Bytecodeに変換されれる。BPのクラスはUBlueprintGeneratedClass（UClassのサブクラス）として登録される。

C++,BPどちらも実行時にはUClass・UFunction・FPropertyのインスタンスとしてメモリ上に存在する

GNames,GObjects

GObjects→ ゲーム内の全UObjectインスタンスへのポインタを持つグローバルな配列。UClassもUFunctionもUObjectを継承しているので、全部ここに登録されている。(詳しくは説明しないが、これはGCなどの為にエンジンが用意している)

GNames→全ての名前（クラス名・関数名・プロパティ名）の文字列が集約されたグローバルなテーブル。UClass等が名前を持つとき、文字列をそのまま持つのではなくFNameというIDだけを持ち、実際の文字列はGNamesで解決する

→この2つのアドレスが分かれば理論上は全てを一気に辿れる
できそうだけど、めんどくさいなぁ...

Built with Opus 4.6

GNames,GObjectsのアドレスを割る

UEのバージョン・ビルドによってオフセットが変わるため、ベースアドレス+オフセットのようなやり方では汎用性がない。
よって、シグネチャスキャンをClaudeによって実装させた。

> モジュールのコード領域を先頭から検索し、

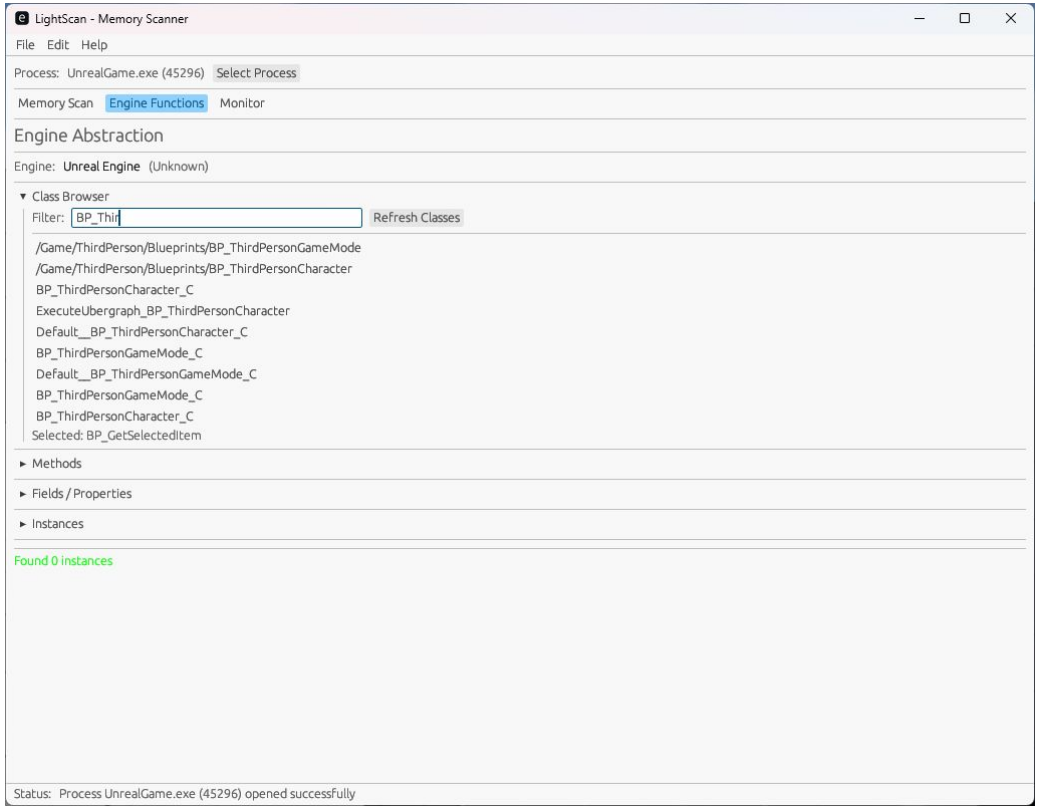
48 8B 05 ?? ?? ?? ??　; mov rax, [rip + offset]　マッチする命令を全て収集する

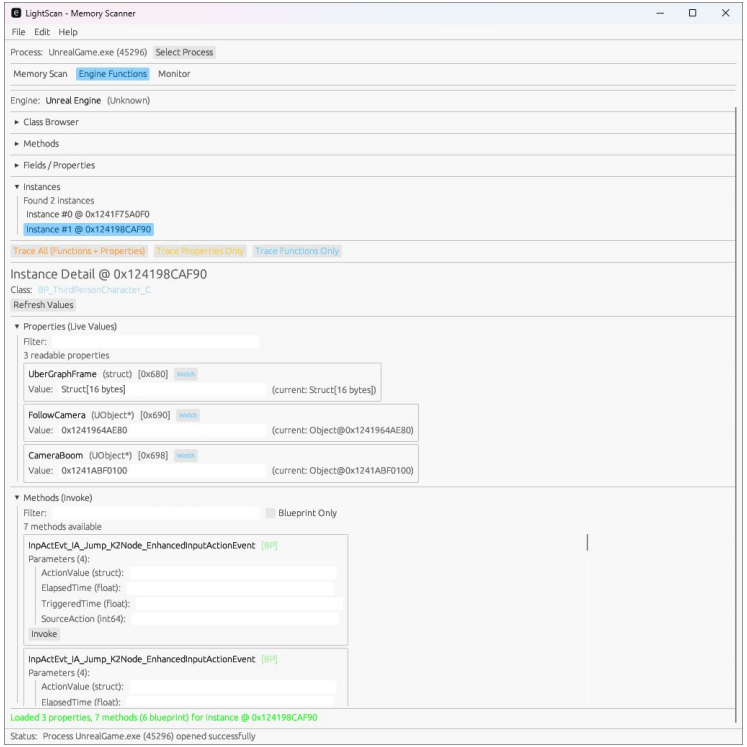
マッチした命令の??部分（オフセット）を読み取り、命令のアドレス ＋ 命令長 ＋ オフセットでGObjects・GNamesの実アドレスを計算する

GNamesならindex 0がNoneという文字列か、GObjectsなら最初のFUObjectItemが有効なUObjectを指しているかなどで総当たりでチェック

こんな感じの処理をClaudeが書いてくれた

スキャン画像


LightScan - Memory Scanner
File Edit Help
Process: UnrealGame.exe (45296) Select Process
Memory Scan Engine Functions Monitor
Engine Abstraction
Engine: Unreal Engine (Unknown)
Class Browser
Filter: BP_Thir
Refresh Classes
/Game/ThirdPerson/Blueprints/BP_ThirdPersonGameMode
/Game/ThirdPerson/Blueprints/BP_ThirdPersonCharacter
BP_ThirdPersonCharacter_C
ExecuteUbergraph_BP_ThirdPersonCharacter
Default__BP_ThirdPersonCharacter_C
BP_ThirdPersonGameMode_C
Default__BP_ThirdPersonGameMode_C
BP_ThirdPersonGameMode_C
BP_ThirdPersonCharacter_C
Selected: BP_GetSelectedItem
Methods
Fields / Properties
Instances
Found 0 instances
Status: Process UnrealGame.exe (45296) opened successfully


LightScan - Memory Scanner
File Edit Help
Process: UnrealGame.exe (45296) Select Process
Memory Scan Engine Functions Monitor
Engine: Unreal Engine (Unknown)
Class Browser
Methods
Fields / Properties
Instances
Found 2 instances
Instance #0 @ 0x1241F75A0F0
Instance #1 @ 0x124198CAF90
Trace All (Functions + Properties) Trace Properties Only Trace Functions Only
Instance Detail @ 0x124198CAF90
Class: BP_ThirdPersonCharacter_C
Refresh Values
Properties (Live Values)
Filter:
3 readable properties
UberGraphFrame (struct) [0x680] Watch
Value: Struct[16 bytes] (current: Struct[16 bytes])
FollowCamera (UObject*) [0x690] Watch
Value: 0x1241964AE80 (current: Object@0x1241964AE80)
CameraBoom (UObject*) [0x698] Watch
Value: 0x1241ABF0100 (current: Object@0x1241ABF0100)
Methods (Invoke)
Filter:
Blueprint Only
7 methods available
InpActEvt_IA_Jump_K2Node_EnhancedInputActionEvent [BP]
Parameters (4):
ActionValue (struct):
ElapsedTime (float):
TriggeredTime (float):
SourceAction (int64):
Invoke
InpActEvt_IA_Jump_K2Node_EnhancedInputActionEvent [BP]
Parameters (4):
ActionValue (struct):
ElapsedTime (float):
Loaded 3 properties, 7 methods (6 blueprint) for instance @ 0x124198CAF90
Status: Process UnrealGame.exe (45296) opened successfully

ProcessEventを勝手に発火させる

先述の通り、BPもC++も関数はUFunctionとして存在している。BP経由で関数呼び出しが発生すると、BPVMはBytecodeを逐次実行し、関数呼び出し命令に到達したとき場合によって[UObject::ProcessEvent](#) を発火させる←おや

```
1 virtual void ProcessEvent
2 (
3     UFunction * Function,
4     void * Parms
5 )
```

`this` — 関数を呼び出す対象の`UObject`インスタンス
`Function` — 実行する`UFunction`
`Parms` — 引数と戻り値を詰めた構造体へのポインタ

https://dev.epicgames.com/documentation/en-us/unreal-engine?application_version=5.5

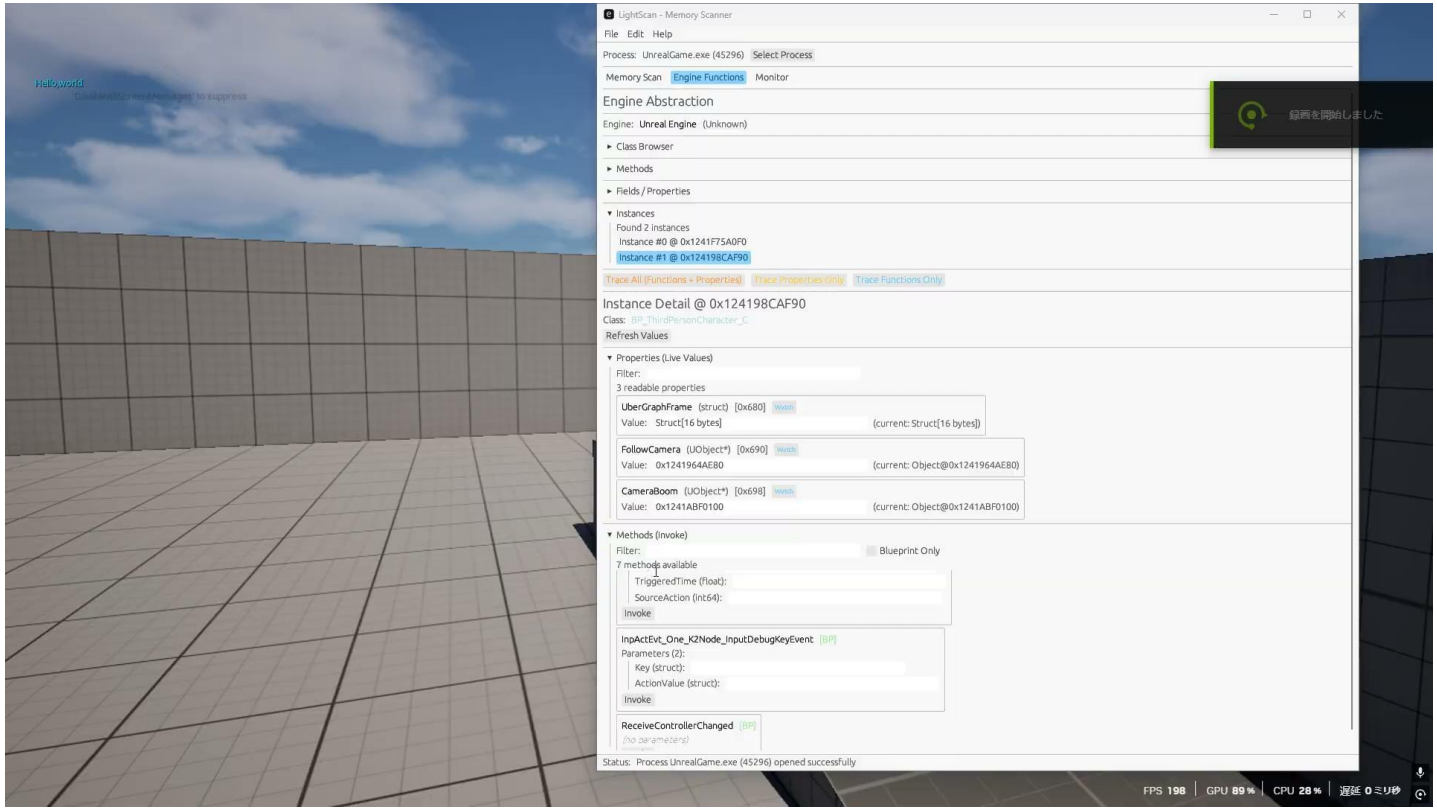
Built with Opus 4.6

ProcessEventを勝手に発火させる

```
mov rcx, UObjectのアドレス      ; 第1引数
mov rdx, UFunctionのアドレス    ; 第2引数
mov r8,  Paramsのアドレス       ; 第3引数
mov rax, ProcessEventのアドレス
call rax
ret
```

GNamesとGObjectsリフレクションの合わせ技により、全てのアドレスが割れているため、VirtualAllocEx→WriteProcessMemory→CreateRemoteThreadのような流れで、ProcessEventを呼び出し任意のBP関数(UFunction)を呼べるようになった。

ProcessEventを勝手に発火させる動画



余談1: ProcessEventをフックするとBPで書かれたActorの関数呼び出しをリアルタイムで観察できて、楽しい。当然リプレイもできる(wip)

余談2: VALORANTはUE4.27ベースだったがこれ(リフレクションによる解析)対策も兼ねて、カスタムPML4エントリによるメモリ隠蔽(shadow baseと形容される)みたいなことをやっているらしい。
私はシステムプログラミングミリしらなので良くわかりません。

https://reversing.info/posts/guardedregions/

まとめ

- パソコンと標的にある程度知見があればClaudeCodeが実装してくれるようになった。感動できる、この喜びを共有したいと思いLTに登壇。
- LTで話したコード群はLightScanとしてGitHubで公開中。

mizuamedesu/
LightScan

A simple memory scanner written in Rust that runs in userland

2 Contributors | 0 Issues | 1 Star | 0 Forks

https://github.com/mizuamedesu/LightScan